Documento de Visão

Versão 2.0

Revisões do Documento

Revisões são melhoramentos na estrutura do documento e no seu conteúdo. O objetivo primário desta tabela é a fácil identificação da versão do documento. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	Observações
1.0	19/03/2026	Maria Fernanda L. R. de Sousa, Maria Clara Moita, Luiz Guilherme e João Gabriel	Foram adicionados no do documento de visão a introdução, com finalidade, escopo, definições e referências, a contextualização do problema e os stakeholders.
2.0	09/04/2026	Maria Fernanda L. R. de Sousa	Foram adicionados no do documento a visão geral do produto, os requisitos, as precedências e prioridades e o manual de arquiteturas

Sumário

1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 5
1.4 Referências 5
1.4.1 Especificações Suplementares 5
2.1 Descrição do Problema 6
2.2 Sentença de Posição do Produto 6

3. Descrição dos *Stakeholders* e dos Usuários 7

3.1 Principais *Stakeholders* e Usuários 7
3.2 Necessidades Chave dos *Stakeholders* e dos Usuários 8

4. Visão Geral do Produto 8
4.1 Perspectiva do Produto 8
4.2 Estimativa de Custo Inicial 9
4.3 Suposições e dependências 9
4.4 Premissas e Dependências 9

5. Requisitos Funcionais e Não Funcionais do Produto 10
5.1 Requisitos Funcionais 10
5.2 Requisitos Não Funcionais 10

6. Precedência e Prioridades 11

7. Manual de Arquitetura 11
7.1 Visão de Implementação 12
7.2 Mecanismo Arquiteturais 12

Documento de Visão

1. Introdução

1.1 Finalidade

A finalidade deste documento é apresentar a visão geral do sistema de detecção de credenciais em repositórios, definindo como os stakeholders enxergam a solução, suas funcionalidades principais e as necessidades que ela busca atender.

O sistema proposto consiste em um scanner automatizado de credenciais e dados sensíveis em repositórios de código, com execução antes do commit, com o objetivo de prevenir vazamentos acidentais de informações críticas.

A principal proposta é ir além dos scanners tradicionais, que utilizam apenas padrões simples, incorporando uma análise mais inteligente baseada em contexto. O sistema será capaz de identificar não apenas segredos explícitos, como senhas e chaves, mas também dados ofuscados ou escondidos, além de analisar o uso dessas informações dentro do código.

A ferramenta será integrada ao fluxo de desenvolvimento por meio de um *pre-commit hook* do Git, atuando de forma preventiva e educativa, alertando o desenvolvedor ou bloqueando o commit quando necessário.

1.2 Escopo

1.2.1 Objetivo Geral

Desenvolver um sistema capaz de identificar e prevenir o vazamento de credenciais e dados sensíveis em repositórios de código, por meio da análise automática de arquivos antes da realização de commits no Git.

1.2.2 Objetivos específicos:

- Expandir os formatos de arquivos a serem analisados: O sistema será capaz de analisar arquivos de código-fonte, arquivos de configuração (.env, .json, .yaml, .xml), dumps de banco de dados (.sql), arquivos de log (.log), arquivos de chave (.pem, .key, .crt) e arquivos compactados (.zip);
- Aprimorar a análise de formatos de arquivos já incluídos em scanners comuns;
- Proporcionar conhecimento com relação a correções e implicações do erro: Proporcionar ao desenvolvedor orientações sobre boas práticas de segurança e correção de vulnerabilidades identificadas;
- Identificar padrões específicos em dados sensíveis;
- Integrar o sistema ao fluxo de desenvolvimento por meio de hooks do Git, permitindo a execução automática antes dos commits;
- Detectar vazamentos em arquivos não convencionais: Ao identificar possíveis dados sensíveis, o sistema deverá gerar alertas ao desenvolvedor, podendo impedir a realização do commit;
- Integrar ao fluxo de desenvolvimento;
- Notificar e orientar o desenvolvedor: Emitir alertas claros e fornecer sugestões de correção para evitar o vazamento de dados sensíveis;
- Reduzir risco de exposição de dados;
- Propor melhorias em relação às ferramentas existentes.

1.2.3 Planejamento de recursos:

- Recursos Humanos
  - Desenvolvimento
  - Testes
  - Documentação

- Recursos tecnológicos
  - Linguagem: Python
  - Versionamento: Git

- Repositório: Github
- Ambiente de desenvolvimento: Visual Studio Code

❖ Recursos computacionais
- Computador pessoal com capacidade para execução de scripts Python e manipulação de arquivos locais.

❖ Tempo
- Planejamento de requisitos
- Desenvolvimento do produto
- Testes
- Documentação

❖ Delimitação do escopo
- O sistema não contempla análise de arquivos multimídia, como imagens e vídeos, nem técnicas avançadas como OCR, sendo essas consideradas possíveis evoluções futuras.

1.3 Definições, Acrônimos e Abreviações

- **Git**: Sistema de controle de versão distribuído utilizado para gerenciar código-fonte.
- **Commit**: Ação de salvar alterações no repositório.
- **Pre-commit hook**: Script executado automaticamente antes da confirmação de um commit.
- **API Key**: Chave utilizada para autenticação em serviços e APIs.
- **Token**: Código de autenticação usado para acesso a sistemas.
- **Base64**: Método de codificação de dados frequentemente utilizado para ofuscação.
- **Entropia**: Medida utilizada para identificar o nível de aleatoriedade de uma string, podendo indicar um possível segredo.

Mais informações poderão ser detalhadas futuramente no glossário do sistema.

1.4 Referências

1.4.1 Especificações Suplementares

- Git. *Git Documentation*. Git SCM. Disponível em: https://git-scm.com/docs. Acesso em: 20 mar. 2026.
- Git. *Customizing Git – Git Hooks*. Disponível em: https://git-scm.com/docs/githooks. Acesso em: 20 mar. 2026.
- OWASP. *OWASP Top 10 – Web Application Security Risks*. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 20 mar. 2026.
- OWASP. *Secrets Management Cheat Sheet*. Disponível em: https://cheatsheetseries.owasp.org/. Acesso em: 20 mar. 2026.
- Lei Geral de Proteção de Dados Pessoais. Lei nº 13.709, de 14 de agosto de 2018. Disponível em: https://www.planalto.gov.br/. Acesso em: 20 mar. 2026.

- GitHub. *Secret Scanning Documentation*. Disponível em: https://docs.github.com/. Acesso em: 20 mar. 2026.
- DATACAMP. *Guia completo sobre Git Hooks: instalação, uso e dicas*. 2025. Disponível em: https://www.datacamp.com/pt/tutorial/git-hooks-complete-guide

2. Contextualização

Atualmente, um dos principais problemas enfrentados no desenvolvimento de software é o vazamento acidental de credenciais e dados sensíveis em repositórios de código. Esse tipo de falha ocorre, na maioria das vezes, por descuido dos desenvolvedores ao incluir informações como senhas, tokens, chaves de API ou arquivos de configuração diretamente no código.
As ferramentas existentes para detecção de segredos apresentam limitações relevantes, principalmente relacionadas à alta taxa de falsos positivos, pois utilizam apenas padrões simples baseados em expressões regulares. Isso faz com que muitas strings sejam identificadas como ameaças, mesmo quando não representam risco real, reduzindo a confiança dos desenvolvedores nessas ferramentas.

2.1 Descrição do Problema

Problemas	• Vazamento acidental de credenciais (senhas, tokens, API keys) em repositórios de código • Alta taxa de falsos positivos em ferramentas atuais de detecção • Dificuldade em identificar segredos ofuscados (Base64, hexadecimal, etc.) • Falta de análise de arquivos não-código (ex: *.env*, dumps, planilhas) • Ausência de verificação eficiente no histórico do repositório • Baixa conscientização dos desenvolvedores sobre boas práticas de segurança
Pessoas Atingidas	• Desenvolvedores de software • Equipes de Segurança da Informação (AppSec) • Empresas e organizações que utilizam repositórios de código • Equipes de DevOps e gestores de TI
Cujo impacto é	• Exposição de dados sensíveis e credenciais • Risco de ataques cibernéticos e acessos não autorizados • Prejuízos financeiros e danos à reputação da organização • Retrabalho para remoção de segredos já versionados • Redução da confiança em ferramentas de segurança devido a falsos positivos
Uma solução bem-sucedida traria	• Prevenção de vazamentos antes do commit • Redução de falsos positivos com análise de contexto • Detecção de segredos explícitos e ofuscados • Maior segurança nos repositórios de código • Conscientização dos desenvolvedores sobre boas práticas • Integração simples ao fluxo de desenvolvimento sem impactar a produtividade

2.2 Sentença de Posição do Produto

Para	equipes de desenvolvimento de software, profissionais de segurança da informação (AppSec) e

	organizações que utilizam repositórios de código
O	scanner inteligente de credenciais em repositórios
Que	identifica e previne o vazamento de dados sensíveis, como senhas, tokens e chaves de API, por meio de análise baseada em padrões e contexto, atuando antes da realização de commits
Diferente de	ferramentas tradicionais de detecção de segredos que utilizam apenas padrões simples (regex) e atuam de forma reativa
Nosso produto	oferece uma abordagem preventiva e mais inteligente, integrando-se ao fluxo de desenvolvimento via hooks do Git, utilizando análise contextual e técnicas como entropia para detectar inclusive informações ofuscadas, além de atuar de forma educativa ao alertar o desenvolvedor e reduzir riscos de vazamentos

3. Descrição dos *Stakeholders* e dos Usuários

3.1 Principais *Stakeholders* e Usuários

Identificação	Responsabilidades	*Stakeholders*
Desenvolvedor de Software	Implementar código, realizar commits e tratar os alertas de segurança gerados pelo scanner antes de enviar o código ao repositório.	Usuário
Líder Técnico (Tech Lead)	Definir as regras de bloqueio, configurar os níveis de sensibilidade do scanner e garantir que a ferramenta seja adotada por toda a equipe.	Stakeholder
Equipe de Segurança (AppSec)	Validar os padrões de detecção (regex/entropia), auditar possíveis falsos positivos e definir os critérios de conformidade de segurança.	Stakeholder
Gestor de TI / Coordenação	Garantir que o projeto minimize riscos de vazamento de dados (LGPD) e validar o impacto da ferramenta na produtividade da equipe.	Stakeholder

3.2 Necessidades Chave dos *Stakeholders* e dos Usuários

No.	Descrição	Prioridade do Cliente (Crítico, Útil e Importante)	Observações
1	Detectar automaticamente credenciais e dados sensíveis (senhas, tokens, API Keys) no código antes do commit	Crítico	Funcionalidade principal do sistema
2	Integrar o scanner ao fluxo de desenvolvimento por meio de hooks do Git	Crítico	Deve funcionar de forma transparente para o desenvolvedor
3	Identificar segredos mesmo quando estiverem ofuscados (ex: Base64 ou strings aleatórias)	Crítico	Uso de análise por entropia e contexto
4	Alertar o desenvolvedor de forma clara e educativa sobre possíveis vazamentos	Importante	Evitar frustração e melhorar adoção
5	Permitir configuração de níveis de sensibilidade e regras de detecção	Importante	Necessidade do Tech Lead e equipe de segurança
6	Minimizar falsos positivos para não impactar a produtividade da equipe	Crítico	Essencial para aceitação da ferramenta
7	Garantir conformidade com boas práticas de segurança e proteção de dados	Importante	Alinhamento com políticas e normas
8	Gerar logs ou relatórios simples para auditoria de segurança	Útil	Apoio para equipe de segurança (AppSec)

4. Visão Geral do Produto

O AnalisadorGit é uma ferramenta de apoio ao desenvolvimento de software que atua diretamente no fluxo de versionamento, utilizando hooks do Git para realizar análises automáticas no código antes e após commits.
O sistema tem como principal objetivo identificar problemas no código, como padrões inadequados, possíveis falhas ou violações de boas práticas, antes que essas alterações sejam registradas no repositório.
A solução funciona de forma integrada ao ambiente local do desenvolvedor, acionando scripts automatizados em momentos específicos do fluxo de trabalho, como durante o pre-commit, garantindo maior controle de qualidade no código versionado.
Dessa forma, o sistema atua como uma camada preventiva, evitando a inclusão de código problemático e contribuindo para a manutenção da qualidade do software.

4.1 Perspectiva do Produto

O AnalisadorGit é um sistema complementar ao Git, operando por meio de hooks locais que permitem interceptar eventos do fluxo de versionamento, como commits e merges.
O produto não substitui o Git, mas atua como uma extensão funcional que adiciona validações automáticas ao processo de desenvolvimento.
Sua arquitetura é baseada em scripts executados localmente, com integração entre:

- Hooks do Git (pre-commit, commit-msg, post-commit, post-merge)

- Scripts desenvolvidos em Python para análise de código
- Configurações locais do ambiente do desenvolvedor

O sistema depende da correta configuração dos hooks no diretório .git/hooks, sendo executado no ambiente local de cada desenvolvedor.
Por não ser versionado automaticamente pelo Git, o produto exige configuração manual em cada máquina, o que impacta sua padronização em equipes maiores.

4.2 Estimativa de Custo Inicial

A estimativa de custo inicial do AnalisadorGit foi baseada na técnica de Pontos de Função, resultando em aproximadamente 37 PF, caracterizando o sistema como de pequeno a médio porte.
Considerando que o sistema foi desenvolvido em ambiente acadêmico, utilizando tecnologias gratuitas como Git e Python, não há custos diretos com licenças ou infraestrutura.
Entretanto, ao considerar o esforço de desenvolvimento, estima-se:

- Tempo de desenvolvimento: entre 2 a 3 semanas
- Esforço total: aproximadamente 60 a 100 horas

Em um cenário profissional, considerando um valor médio de R$ 50 a R$ 80 por hora de desenvolvimento, o custo estimado do projeto poderia variar entre R$ 3.000 e R$ 8.000.
Esse valor pode variar de acordo com a complexidade das regras de análise implementadas e o nível de automação desejado.

4.3 Suposições e dependências

Para o funcionamento adequado do sistema, foram consideradas as seguintes suposições:

- Os desenvolvedores irão configurar corretamente os hooks do Git em seus ambientes locais
- O ambiente de execução possui suporte à linguagem Python
- Os repositórios analisados seguem a estrutura padrão do Git
- Os usuários não irão desativar os hooks durante o uso normal do sistema

O sistema possui as seguintes dependências:

- Git instalado e configurado
- Ambiente de execução Python
- Permissão de acesso ao repositório analisado
- Sistema operacional compatível com execução de scripts shell

Além disso, o funcionamento do sistema pode ser impactado por mudanças na estrutura interna do Git ou na forma como hooks são executados.

4.4 Premissas e Dependências

O desenvolvimento do AnalisadorGit baseia-se nas seguintes premissas:

- Os hooks do Git serão utilizados como mecanismo principal de automação
- A análise do código será realizada localmente, sem necessidade de servidores externos
- O volume de código analisado será compatível com o tempo de execução dos hooks, evitando atrasos significativos no commit

As dependências do sistema incluem:

- Estrutura de diretórios do Git (.git/hooks)
- Scripts de análise implementados em Python
- Permissões de execução dos arquivos de hook

Uma limitação importante é que os hooks não são versionados pelo Git, o que significa que sua configuração não é automaticamente compartilhada entre desenvolvedores. Isso pode comprometer a padronização e a eficácia do sistema em ambientes colaborativos.

5. Requisitos Funcionais e Não Funcionais do Produto

5.1 Requisitos Funcionais

RF01 – Configurar hooks do Git
O sistema deve permitir a configuração de hooks no diretório .git/hooks para automação do processo de análise.

RF02 – Executar análise no pre-commit
O sistema deve executar automaticamente a análise dos arquivos modificados antes da realização do commit.

RF03 – Identificar arquivos modificados
O sistema deve identificar quais arquivos foram alterados no commit.

RF04 – Analisar código-fonte
O sistema deve analisar o conteúdo dos arquivos modificados utilizando scripts em Python.

RF05 – Detectar problemas no código
O sistema deve identificar padrões inadequados, erros ou possíveis vulnerabilidades no código.

RF06 – Bloquear commit
O sistema deve impedir a conclusão do commit caso sejam detectados problemas.

RF07 – Permitir commit válido
O sistema deve permitir a conclusão do commit quando nenhuma inconsistência for encontrada.

RF08 – Validar mensagem de commit
O sistema deve verificar se a mensagem de commit segue um padrão definido.

RF09 – Exibir mensagens ao usuário
O sistema deve informar ao usuário o resultado da análise (erro ou sucesso).

RF10 – Registrar informações pós-commit
O sistema deve registrar dados do commit após sua conclusão.

RF11 – Processar eventos de merge (pull)
O sistema deve executar ações após operações de merge, como atualização de métricas.

5.2 Requisitos Não Funcionais

RNF01 – Desempenho
A análise deve ser executada rapidamente, sem causar atrasos significativos no processo de commit.

RNF02 – Usabilidade
O sistema deve fornecer mensagens claras e compreensíveis ao usuário.

RNF03 – Portabilidade
O sistema deve funcionar em diferentes sistemas operacionais compatíveis com Git e Python.

RNF04 – Segurança
O sistema deve evitar a exposição de dados sensíveis durante a análise do código.

RNF05 – Confiabilidade
O sistema deve executar as análises de forma consistente, sem falhas durante o processo de commit.

RNF06 – Manutenibilidade
O código do sistema deve ser organizado e permitir fácil atualização das regras de análise.

RNF07 – Escalabilidade
O sistema deve suportar análise de projetos com diferentes tamanhos e volumes de arquivos.

RNF08 – Configurabilidade
O sistema deve permitir adaptação das regras de análise por meio de arquivos de configuração.

RNF09 – Compatibilidade
O sistema deve ser compatível com a estrutura padrão de repositórios Git.

RNF10 – Limitação técnica
O sistema deve considerar que hooks do Git não são versionados e podem ser desativados pelo usuário.

6. Precedência e Prioridades

A precedência e prioridade dos requisitos foram definidas com base na dependência funcional entre as atividades do sistema e na importância de cada funcionalidade para o funcionamento do AnalisadorGit. A precedência indica a ordem em que os requisitos devem ser implementados ou executados, considerando que algumas funcionalidades dependem da conclusão de outras. Já a prioridade define o nível de importância de cada requisito, sendo classificada em alta, média ou baixa, de acordo com seu impacto no funcionamento do sistema. Funcionalidades essenciais, como a execução da análise no pre-commit e o bloqueio de commits com problemas, foram classificadas como de alta prioridade, pois representam o núcleo do sistema. Já funcionalidades complementares, como geração de relatórios e registro pós-commit, foram classificadas como média ou baixa prioridade.

ID	Requisito	Precedência	Prioridade
RF01	Inserir repositório	Nenhuma	Alta
RF02	Validar repositório	RF01	Alta
RF03	Configurar hooks do Git	Nenhuma	Alta
RF04	Executar análise no pre-commit	RF03	Alta
RF05	Analisar arquivos modificados	RF04	Alta
RF06	Bloquear commit com erro	RF05	Alta
RF07	Validar mensagem de commit	RF03	Média
RF08	Gerar relatório de análise	RF05	Média
RF09	Registrar dados pós-commit	RF06	Baixa
RF10	Processar eventos de merge (pull)	RF06	Baixa

7. Manual de Arquitetura

O Manual de Arquitetura do sistema AnalisadorGit descreve a organização estrutural da aplicação, seus componentes principais e os mecanismos utilizados para garantir seu funcionamento. O sistema foi projetado com foco na automação da análise de código durante o fluxo de versionamento, utilizando hooks do Git e scripts em Python para execução das regras de validação. A arquitetura do AnalisadorGit se destaca por sua integração direta com o fluxo de desenvolvimento, atuando de forma transparente e preventiva, sem necessidade de ferramentas externas complexas, o que a

torna leve, eficiente e de fácil adoção.

7.1 Visão de Implementação

A implementação do AnalisadorGit é baseada em uma arquitetura leve e modular, composta por scripts locais integrados ao fluxo do Git por meio de hooks.
O sistema não possui uma interface gráfica tradicional, sendo executado diretamente no ambiente de desenvolvimento do usuário, de forma transparente durante operações como commit e merge.

A estrutura de implementação é organizada da seguinte forma:

- **Hooks do Git:** Localizados no diretório .git/hooks, responsáveis por interceptar eventos como pre-commit, commit-msg e post-commit
- **Scripts em Python:** Responsáveis pela lógica de análise, processamento dos arquivos e validação das regras
- **Scripts Shell:** Utilizados para acionar os scripts Python a partir dos hooks
-

O fluxo de execução ocorre da seguinte maneira:

1. O desenvolvedor realiza alterações no código
2. Ao executar um commit, o hook pre-commit é acionado
3. O hook executa um script que chama o analisador em Python
4. O sistema analisa os arquivos modificados
5. Caso sejam encontrados problemas, o commit é bloqueado
6. Caso contrário, o commit é concluído normalmente

A implementação prioriza simplicidade e eficiência, utilizando ferramentas nativas do ambiente de desenvolvimento, sem necessidade de servidores externos ou infraestrutura adicional.

7.2 Mecanismo Arquiteturais

O sistema AnalisadorGit utiliza diversos mecanismos arquiteturais para garantir seu funcionamento eficiente e integrado ao fluxo de desenvolvimento. Entre os principais mecanismos, destacam-se:
• Hooks do Git:
Permitem a execução automática de scripts em eventos específicos do versionamento, como antes e depois de commits, garantindo a integração do sistema ao fluxo de trabalho do desenvolvedor.

• Execução Local:
O sistema opera inteiramente no ambiente local do usuário, reduzindo dependências externas e aumentando a velocidade de execução.

• Integração com Python:
Scripts em Python são utilizados para realizar a análise de código, permitindo maior flexibilidade e facilidade na implementação de regras de validação.

• Validação Preventiva:
O sistema atua antes da confirmação do commit, impedindo que código inadequado seja versionado, o que melhora a qualidade do software desde as etapas iniciais.

• Modularidade:
A separação entre hooks, scripts de execução e lógica de análise permite manutenção e evolução do sistema de forma independente.

• Configurabilidade:
As regras de análise podem ser adaptadas conforme a necessidade do projeto, permitindo maior flexibilidade de uso.

• Tolerância a Falhas:
Em caso de erro na execução dos scripts, o sistema pode interromper o commit ou informar o

usuário, garantindo controle sobre o processo.

• **Limitação Arquitetural:**
Como os hooks do Git não são versionados, sua configuração depende do ambiente local de cada desenvolvedor, o que pode impactar a padronização em equipes.